

Distributed Deep Learning Inference at Edge Devices

FYP Progress Report - Group 44

160170B	K.M.B.G.B Gunarathne
160477X	M.V.R.Perera
160628M	T.M.C.P Thennakoon

Table Of Contents

Table Of Contents	2
1. Introduction	4
1.1 Problem Statement	6
1.2. Research Objectives	7
2. Literature Review	7
2.1 Background	8
2.1.1 Background of DNN	8
2.1.2 Fully Connected Neural Network	8
2.1.3 Model Parallelism	10
2.2 Related Work	10
3. Methodology	13
3.1 Proposed Methodology	13
3.1.1 Model Parallelism Based Partitioning	13
3.1.2 Communication and Task sharing	15
3.2 System Design	16
3.2.1 Model Optimizer	16
3.2.2 Inference Engine	17
3.2.3 Fault tolerance mechanism	18
3.2.4 Model Partitioning Schemes	19
3.2.4.1 Layer wise partitioning	19
3.2.4.2 Other partitioning methods	19
3.2.5 Communication demands	20
3.2.6 User application support	20
3.2.7 Programming language for the implementation	20
4. Progress	22
4.1 Introduction	22
Open Neural Network Exchange	24
4.2 Experimental evaluation	25
4.2.1 Model Optimizer for YOLO v3	25
4.2.1.1 What is YOLO v3	25
4.2.1.2 Architecture of the Model	26
4.2.1.2.1 Types of Layers	26
4.2.1.2.2 Features of the Model	26
4.2.1.2.3 Output	27
4.2.1.2.4 Make predictions using outputs	28
4.2.1.3 Pre Trained YOLO Model	28

4.2.2 Model Optimizer for ONNX	31
4.2.2.1 Alexnet	31
4.2.2.2 Alexnet architecture	31
4.2.2.3 Steps for reading data inside model	32
4.2.3 Model Optimizer for tensorflow models (.pb file)	32
4.2.3 Building a Raspberry Pi Cluster	33
4.2.4 Distributing a Convolution Layer	35
4.2.4.1 Method 1	36
4.2.4.2 Method 2 : Threaded Distribution	39
4.2.4.3 Comparison between method 1 and 2	40
4. Conclusion/Summary	43
References	44

1. Introduction

Deep neural networks are becoming an important tool in modern computing applications. Deep Neural Networks (DNNs) and Convolutional Neural Networks (CNNs) are heavily researched domains and are widely used in large scale data processing applications due to their higher classification accuracy compared to other methods[13]. In this context there are specialized deep neural networks introduced for various tasks. And also a lot of data and computation power is required to train/use these models.

Edge computing and IoT applications have emerged with the requirement of collecting and processing large sets of data locally within devices, thus, ensuring privacy and low latency. However, these edge devices are resource constrained and have less computational power, memory and energy compared to modern computers and servers. These edge devices may collect large amounts of sensory data where it needs to be processed and results computed in real-time for mission critical applications. Deploying Deep/Convolutional Neural Networks (DNNs & CNNs) applications on such mission critical resource constrained devices is a challenging task due to discussed conditions above. DNN/CNN has two phases which are called training and inference. Training of a neural network requires a large set of data and huge computational effort. Once the model is trained and ready it can be fed previously untrained input data and will return results with identified or classified patterns in the input data. This phase is called inference. Even Though it does not require huge computational power compared to training, executing inference tasks in resource constrained edge devices is still a challenging task [1].

Performing inference tasks of deep learning applications on edge devices ensures privacy of the input data and prevents latency and availability issues compared to a cloud solution. As most edge devices are resource constrained (eg : memory , computational) they cannot store and execute an entire Deep Neural Network(DNN). According to paper [12], One possible solution is to distribute the DNN across multiple edge devices. Layers in a neural network

need to be carefully partitioned to balance the amount of computation and memory usage on each resource constrained edge device and to reduce the communication among devices.

In this research we are focusing on the execution of the DNN inference part in resource constrained edge devices like Raspberry Pi boards. We intend to research and propose a novel approach to partition and distributively execute inference calculation on multiple edge devices which are connected in a local network. Furthermore, we plan to investigate how to efficiently reduce the communication demands among the devices.

1.1 Problem Statement

IoT devices have been facing the issue of low resources (e.g. computational capacity, memory, energy) since the beginning of the IoT era [10]. Thus, IoT applications that need to use computationally intensive technologies like augmented reality, video processing, cognitive assistance and machine vision are unable to get the best performance and real-time experience with these constraints.

Performing inference tasks of deep neural networks on IoT edge devices requires considerable computational power and memory space which are usually limited on IoT devices. Relatively high-end IoT boards such as Raspberry Pis also have limited resources eg: 1GB - 2GB RAM , 1GHz - 1.5GHz processor etc. (Depends on the versions.) Therefore, storing and execution of a large complete Deep Neural Network in a single device is an infeasible task. Even a medium scale DNN may present intolerable delays in executing inference calculations in these devices. The most common solution to this problem is cloud-based computation. This however introduces availability, privacy and latency issues. Local computing ensures the privacy of sensitive data and results in low latency.

When there are a large number of layers and also when there are a large number of neurons in one layer of a fully connected neural network or when there are computationally intensive convolutional layers, the resources of a Raspberry Pi device will either be not enough or it will take an unacceptable amount of time to compute the result.

Deep Learning Software Framework	MobileNet v1 (ms) Benchmark value	MobileNet V2 (ms) Benchmark value	Inception model that we tested (s)
TensorFlow Lite	271.5	379.6	24
TensorFlow	480.3	654.0	32

Table 1. A comparison of image classification time taken on a Raspberry Pi device

As shown in Table 1 above, Tensorflow optimised models like MobileNet v1 and v2 have acceptable low time consumption to complete execution, but we have tested a custom model (Inception model) an object detection library on our Raspberry Pi 3 B V1 setup, and it took an unacceptable amount of time compared to the Tensorflow given benchmark values [11].

We identified a lack of conducted research in this area to provide model partitioning and distributed execution in resource constrained IoT devices (etc. raspberry pi, mobile devices..) considering computational , memory and communication demands. So our problem is to find a more efficient way to run large Deep Neural Networks and improve the overall performance of distributed Deep Learning Inference on low resource IoT edge devices.

1.2. Research Objectives

The final intended outcome of this research is to propose a novel approach to build communication and memory aware partitioning and distribution schemes for fully connected neural network layers of deep neural networks among suitable resource constrained edge devices. Furthermore, a proof of concept prototype for distributed DNN/CNN execution is planned to be implemented.

- a. Discover model parallelism based partitioning scheme for fully connected neural networks.
- b. Propose a task distribution and task sharing mechanism among suitable resource constrained edge devices. (Eg: Raspberry Pi, mobile devices)
- c. Implement a proof of concept DNN/CNN partitioning and distributed processing application using python.
- d. Compare performance of the developed system with existing systems.

2. Literature Review

In this section we first present some background information on neural networks. Then we proceed to discuss related work on distributed deep learning inference, challenges faced and the issues which are not yet addressed in the area.

2.1 Background

2.1.1 Background of DNN

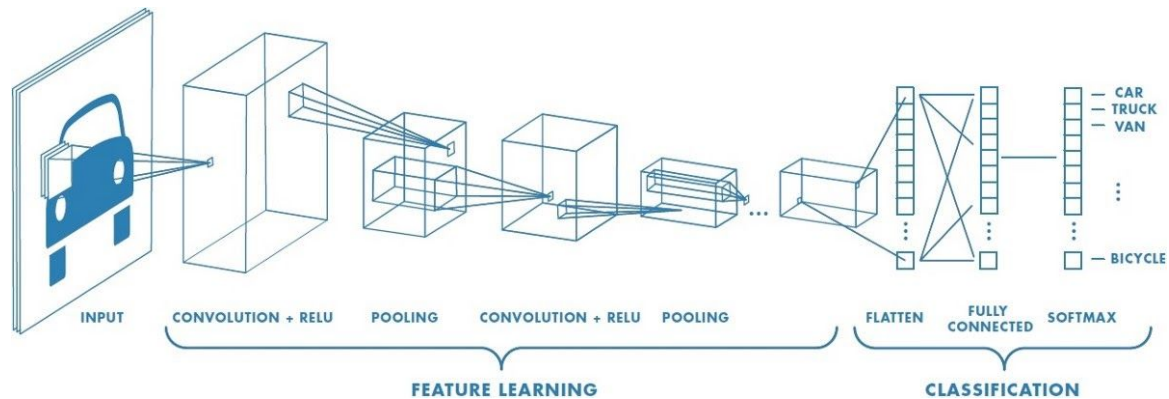


Figure 2.CNN architecture

The Figure 2 shows an overview of an Image classification Deep Neural Network with convolutional layers and fully connected layers. Convolutional layers are used to extract features (ie: edges, lines, curves, etc) in the given input. The fully connected network is used to classify the images, based on recognized features of the input data. Within these layers heavy computational tasks are executed while propagating through the network.

Within these convolutional layers it applies multiple pre-learned filters to their input to produce a set of feature maps which extract some features from the input. There can be many filters between each convolution layer which increase the computational and memory demand. Pooling layers are used to shrink the size of input feature maps.

2.1.2 Fully Connected Neural Network

Fully connected neural networks are usually used for recognizing and classification tasks. Therefore, convolution layers followed by fully connected layers are used in image recognition and classification. Fully connected layers consist of a number of neurons which

have a pre learned bias value and activation function. These neurons are connected with all previous layer's neurons and the next layer's neurons where these connections have pre-learned weights.

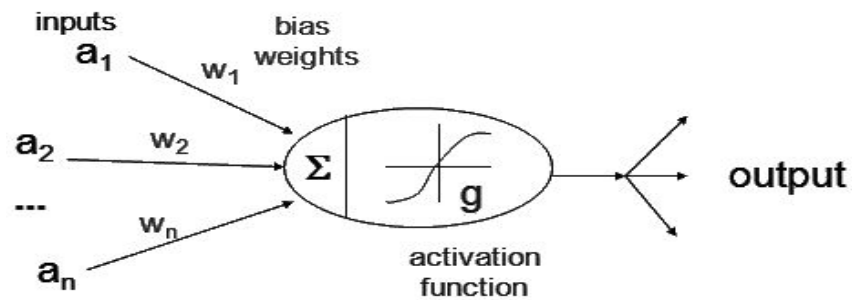


Figure 3. Activate Function process inside a neuron

The calculation inside each neurons can be described as follows

$$f\left(b + \sum_{i=1}^n a_i w_i\right)$$

Where

b = bias

a = input to neuron

w = weights

n = the number of inputs from the incoming layer

i = a counter from 0 to n

f = an Activation function

Therefore each calculation requires all input values of neurons for all connected non-zero weights. Therefore one possible solution will be to partition each calculation of neurons by sharing all input data and weight data to all devices. However, it will be not memory efficient as well as communication efficient.

2.1.3 Model Parallelism

Model parallelism is used to partition the neural network layers in vertical direction and assign each device with a subset of these neurons to execute calculations. As this method splits the neural network this method can be possibly applied to deploy large neural networks which normally can not be deployed in a single device. Furthermore, model parallelism is used for training large neural networks in GPU clusters. Therefore applying this method for the inference task is sound feasible and promising.

2.2 Related Work

Improving Deep Learning inference performance to overcome challenges it has faced is an intensively researched area [1]. Inference is computationally costly when it comes to high dimensional inputs. Therefore, a lot of calculations have to be done on that input data. One common solution was to use cloud computing as a solution, as they can use cloud resources for computation. But moving the data from the source to cloud introduced some challenges such as latency, scalability and privacy [1]. When it comes to real time applications, latency incurred by additional queuing and propagation delays of the network is not acceptable. Unpredictability of the remote cloud server status is also a problem. Uploading all the data from the source to the cloud is inefficient too. Sometimes there may be privacy issues when uploading data. Therefore, to avoid those challenges edge computing came into the play. Edge computing reduced the latency by doing computations in close proximity to the end devices. A hierarchical structure and a local edge server can be used to avoid privacy issues. But edge computing also introduced new challenges. One of the main challenges is using the less powerful edge computing devices for the high resource requirements in deep learning applications. An edge device may be an edge server with a GPU or a mobile device or it may be even a Raspberry Pi device. Another challenge was to coordinate with edge devices to produce an output.

As mentioned in [1], there are three approaches for Deep learning inference in edge computing scenarios.

1. All computations done on the end device.
2. Edge server computation.
3. Computation across edge devices.

In the first method, the model is located at the edge device. As the edge devices are not capable of doing heavy computation and memory intensive tasks, those models need to be designed with less parameters. Otherwise, later the parameters should be compressed or unnecessary parameters should be removed. In DeepIoT [2], the authors present a pruning method for DNN models used in IoT devices. But this may have an impact on the results by reducing the model accuracy. Another sub method of this is where application-specific hardware is used for solutions in order to speed up the process and save energy [1].

Though all those methods are considered, the first method is still challenging because of the resource limitations such as power, computation and memory [1]. Because of that some researchers tend to use the second method which involves an edge server. As done in the deep learning framework presented by the authors of paper [3], where they offload the whole DNN to the edge server for computations. But sometimes it is helpful to do data preprocessing to reduce the communication time with the edge server. In [4], they use removing blurry images, and cropping to take only the required parts of images as preprocessing techniques. A main problem faced here is resource management in the edge server as multiple end devices may be using the same shared edge server. Because of that tradeoff between latency, accuracy and other performance metrics need to be considered. Some solutions such as Mainstream [5] use transfer learning, where common lower layers of the DNN model are used for multiple applications to achieve the above mentioned tradeoff.

Although the second method increases the speed of inference, there is a third method more efficient than the above method, where intelligent offloading is considered. Solutions done in this method face challenges such as taking a decision whether to offload the entire DNN or not (depends on size of the data, hardware abilities, DNN model, network quality), and which fractions of a DNN computations should be offloaded [1]. When using DNN partitioning, some solutions have used layer-wise partitioning as in Neurosurgeon [6] and some have used input-wise partitioning as in MoDNN [7] and DeepThings [8]. Input-wise partitioning also

has a problem of increased data dependency. Some solutions such as DDNN [9] are using a hierarchy of cloud, edge servers and end devices to model partitioning. Cloud is used only if total processing time is reduced, because of the available powerful resources and increased privacy risk.

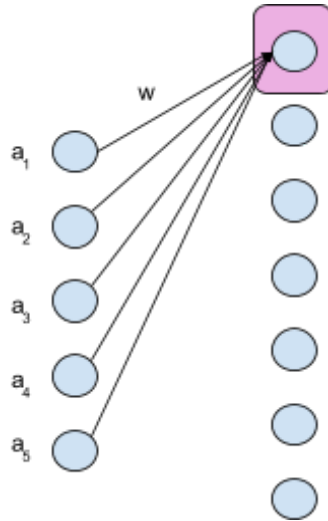
Based on our literature survey, we saw a lack of research done in the area of distributed deep learning inference across multiple helper edge devices without using a remote cloud. Latency and task management are the main issues which are not much addressed in the existing solutions. Latency is caused by less efficient load balancing heuristics and dynamic task sharing systems which we will be improving in our solution. Using fine-grained partitioning of the DNN executions across edge devices, considering their resource availability and network condition seems like a promising method for our solution according to the research.

3. Methodology

3.1 Proposed Methodology

Based on the literature review, we identified that offloading deep learning inference computation across edge devices as a suitable approach. We first describe how we plan to partition the inferencing tasks based on model parallelism. Then, the planned task sharing mechanism among edge devices will be described.

3.1.1 Model Parallelism Based Partitioning



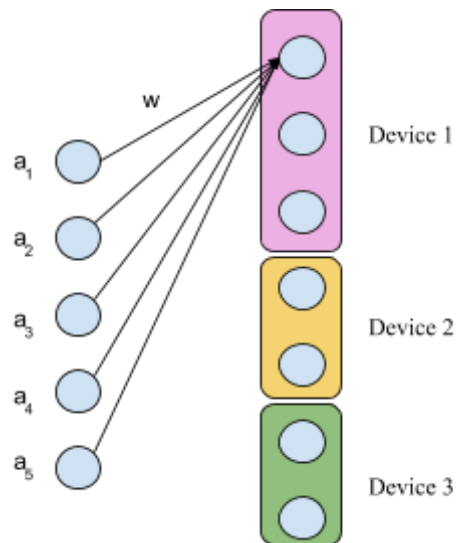
As shown in the above figure the partitioning can be obtained by the output partitioning of the network. By this method the above two layers can be partitioned into seven independently executable tasks. Each task will be added to a task queue in the main device. Also by partitioning this way we can ensure that even a single device can work without any worker devices.

This partitioning enables the ability to distribute the workload among a varying number of worker devices depending on their availability. There will be two configuration options for the distribution method.

1. When the number of worker devices in the network is fixed.

2. When the number of worker devices in the network is dynamic.

If the number of worker devices are fixed and known, the main device will distribute tasks evenly among all devices as shown in the below figure.



If the number of worker devices are dynamic, the main device will only distribute tasks in the task queue when a task sharing request is received. Otherwise, it will continue on executing all remaining tasks in the queue.

3.1.2 Communication and Task sharing

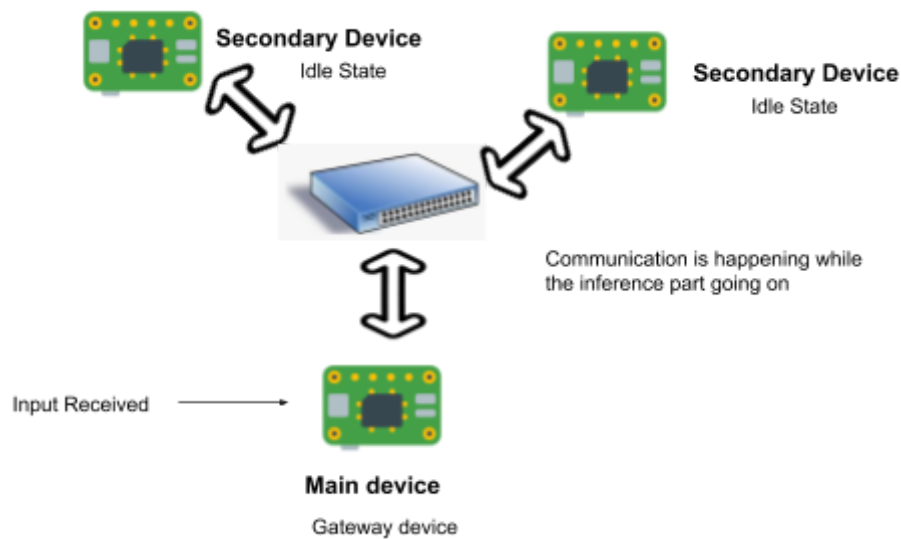


Figure 4. Raspberry pi Communication cluster

As shown in the above figure (figure 4) the main device (the gateway device) will receive input data through its sensors. In this case we assume that there are two other devices on the same network which are in idle state or doing less computationally expensive tasks. These secondary devices also can be changed to a main device (the gateway device) dynamically when it receives an input. All three devices have trained Image Classification model and the partitioning and sharing services running in the background.

The main device which captured the image will run the partitioning program to partition the neural network and input data to separately executable tasks and it will add them to a task queue. Then the main device will start the inference execution by taking one task by one from the queue. Meanwhile, it will broadcast a task sharing request among the network and secondary devices which are in idle state will capture this request and they will notify the main device that they are available to participate. The main device will then start sending a set of tasks from the queue to secondary devices. When the secondary devices have finished the calculation in each neuron they will update the main device with results and request more

tasks from the queue until it is empty or the execution is interrupted due to its own requirements of executing its own process.

3.2 System Design

Based on the above plan we designed the system architecture of the proposed framework.

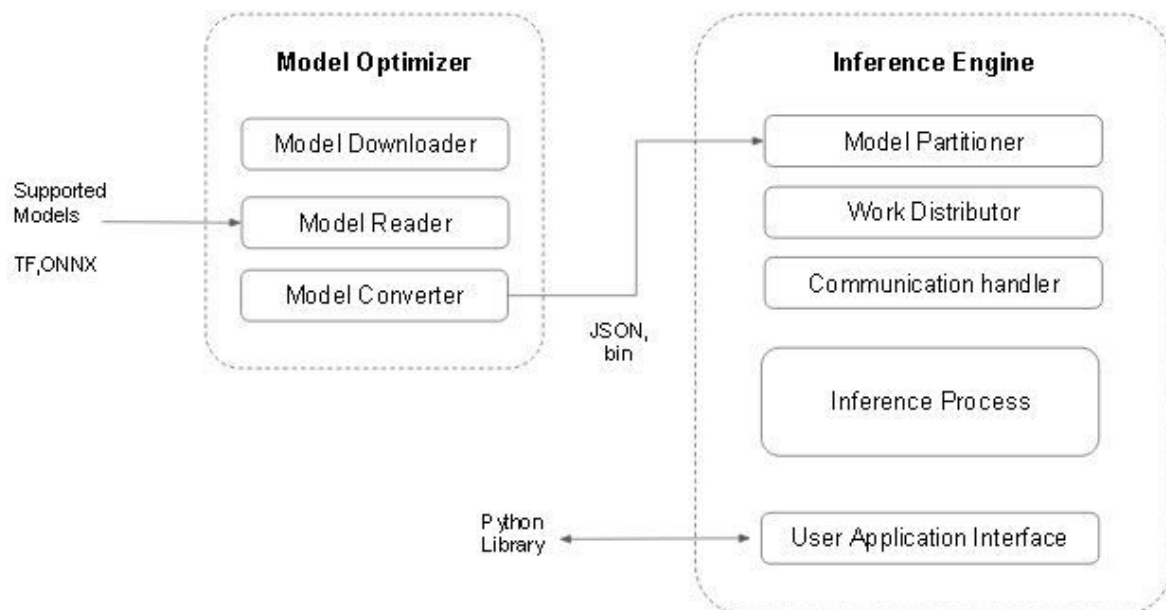


Figure 5. System Design of the proposed Distributed Inference Framework

3.2.1 Model Optimizer

Model optimizer is used to convert external, pre-trained models into our own format. We have to define which models that this tool supports and what kind of layers are supported. These constraints will be defined based on the implementation of the distributed inference engine. Initially we planned to select three widely used models from three different frameworks and build up the model optimizer to support those selected models and their layer types.

Model optimizer is designed to convert pre-trained models into a more optimized format which will be suitable for our distributed inference engine. In other words, conversion can be done in a way that will make it easy to partition the model according to our distribution scheme. Also pre-converting the models helps to reduce the execution time of the framework at runtime as we don't have to handle various types of formats at runtime.

According to our assumption each computational device needs to be set up before using it. Therefore once setup is completed each device will keep the converted model topology and weight data in separate files created by model optimizer. Therefore it just needs to directly read those files at runtime without wasting time to convert each time it runs.

3.2.2 Inference Engine

Distribution of work will take place under two categories: Static distribution and Dynamic distribution.



Figure 6. Task Distribution methods

Based on the type of the setup, the Model partition will decide how to allocate works to each and every available device.

In static the distribution, There is a fixed number of backup devices to support the main device which takes the input. In such a setup work will be partitioned equally among available devices.

In a dynamic setup, every device will be considered as an input taking device. However in the initial setup and networking, we will define the number of devices in the network, So each device knows the maximum number of devices that can become ideal at a given time. Every device will partition the work based on the maximum number of devices in the network and prioritize those tasks considering the order of execution if a single device has to compute the task. Then the main device will start working from the highest priority task. Communication handler will broadcast a task sharing request to the network. If any of the devices is in the idle mode it will communicate with the main device to take the next priority task.

3.2.3 Fault tolerance mechanism

After an idle device has taken a task from another device, it can be interrupted by its own receiving input.

Considering the network failures and task interruptions that can happen, some of the distributed tasks may not get fulfilled by the time it should be completed. Therefore we have applied a fault tolerance mechanism to the system design to continue the work without having any issue.

When a device becomes idle, it will notify other devices that it is available to work. Working device at that time will see this notification and will check if there are any available tasks that can be distributed to idle devices. If all the parts that can be assigned to a secondary device are already assigned, Then the main device will re-assign in the oldest unfinished task to this idle device.

3.2.4 Model Partitioning Schemes

3.2.4.1 Layer wise partitioning

Most available inference engines use this method as it is simple and flexible regardless of the model architecture. Also this method is effectively applicable in GPU or multi core processing.

Device 1 will work on the 1st layer and pass results to the second device to work on the second layer.

Device 1 will take the next input and start working on the 1st layer while Device 2 will complete the first task and pass to the 3rd device. This process will continue until each node is working on a specific layer of different data

3.2.4.2 Other partitioning methods

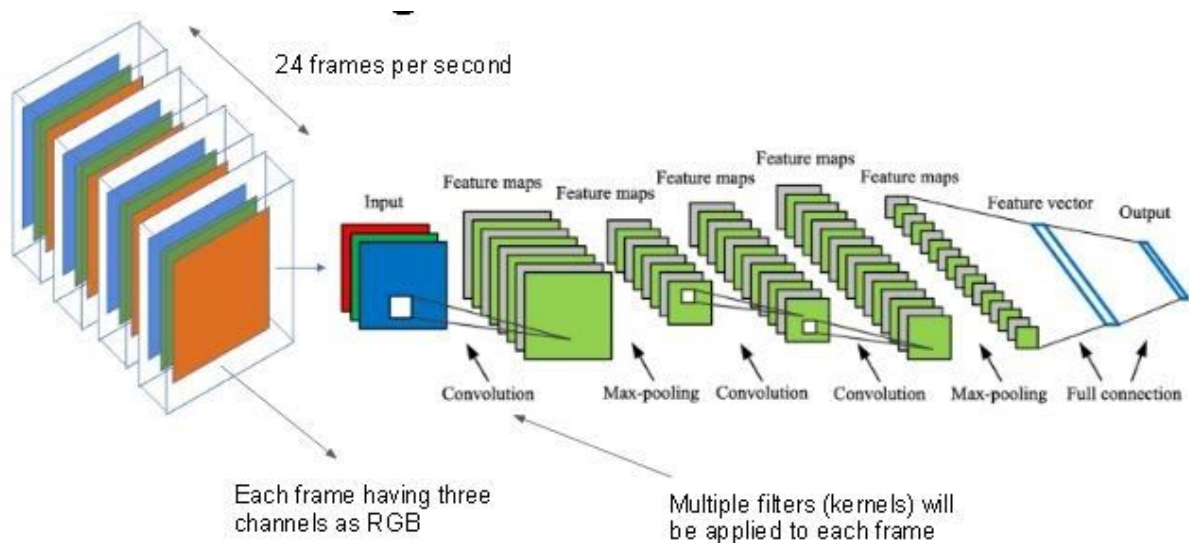


Figure 7. Example Convolutional Neural Network

According to the above figure, We can partition the model

By assigning set of frames to each device (ex : 6 frames per device among 4 devices)

By assigning each channel of a single frame to each device

By assigning set of filters(kernels)

We are planning to implement and try evaluating these methods, Also we hope to come up with more partitioning methods that can be applied.

3.2.5 Communication demands

According to our assumption each device is pre configured. Therefore each device has the model topology and weight data. Therefore there is no need to transfer weight data or model data among devices at runtime.

However we will need to communicate the input data and output results. And instruction data which will indicate which part of the network needs to be calculated by each device.

Other than that work sharing request and other system related communications will happen.

3.2.6 User application support

As our proposed system can not directly work with an external program ie: User's python script, We need to provide an interface to get data and return results to the user application. Therefore we have planned to build a support library.

3.2.7 Programming language for the implementation

Initially we decided to use low level language like C to implement the system hoping it can provide higher performance. However considering the highly optimized library support like numpy library, we may not be able to obtain such performance in C language as we have to implement functionalities that are provided by these optimized libraries. Therefore we decided to use python for the implementation of the proposed system.

4. Progress

4.1 Introduction

We searched for computer vision models which are widely used by developers and researchers. We came up with multiple model collections aka modelzoos available on the internet related to computer vision domain.

There we came with few kinds of deep learning frameworks which are commonly used to build and run these models. For example,

1. Tensorflow (<https://www.tensorflow.org/>)
2. TensorFlow Lite (<https://www.tensorflow.org/lite>)
3. Caffe (<https://caffe.berkeleyvision.org/>)
4. Pytorch (<https://pytorch.org/>)
5. MXnet (<https://mxnet.apache.org/>)
6. Keras (<https://keras.io/>)
7. Darknet (<https://pjreddie.com/darknet/>)
8. ONNX (<https://onnx.ai/>)

Among these frameworks Tensorflow , Keras and Pythorch are most widely used.

Each of these frameworks provide many computer vision related models. And those models can be trained and used for the inference using the inference engines of those frameworks.

Some of the widely used computer vision models are,

1. Darknet YOLO
2. Tensorflow Inception

Furthermore, these models has two phases as

1. Training phase
2. Inference phase

Training phase is used to feed labeled data to the model and let it optimize its weight and bias matrixes. Once the model is trained it can be presented with previously unseen/untrained data and let it provide appropriate results itself.

In this research we are only focused on the inference phase where we use pre trained models in real world applications.

Most of these model developers/original researchers provide pre trained models which users can directly use in the inference process without doing their own training. These trained models have two separate parts which can reside in a single file or which can be separate two files

1. Model architecture
2. Weight file

Model architecture file contains the details like what are the layers in convolutional neural networks, what are the dimensions of filters , how the layers are ordered, architecture of the fully connected layers, etc.

Weight file contains the weight values of each filter and neurons in the network.

Models trained with various frameworks provide these files in various file extensions. For example YOLO model comes with .cfg file and a .weight file while tensorflow models comes with a .pd file, Likewise each of those frameworks use their own file types to provide trained models.

Therefore, we have to make it possible to support as many types of models in our proposed framework to make it useful.

Open Neural Network Exchange

While searching for a method to support as many models, we found Open Neural Network Exchange aka ONNX. It is an open standard for machine learning interoperability. ONNX can be used to convert pre-trained models into onnx format. Therefore, if we can support ONNX on our framework we can use it as a gateway to use many other pre-trained models.

Finally we came up with the idea of developing our own model converter tool which can read and convert external models into our own format which is fully optimized for our distributed inference engine.

4.2 Experimental evaluation

4.2.1 Model Optimizer for YOLO v3

4.2.1.1 What is YOLO v3

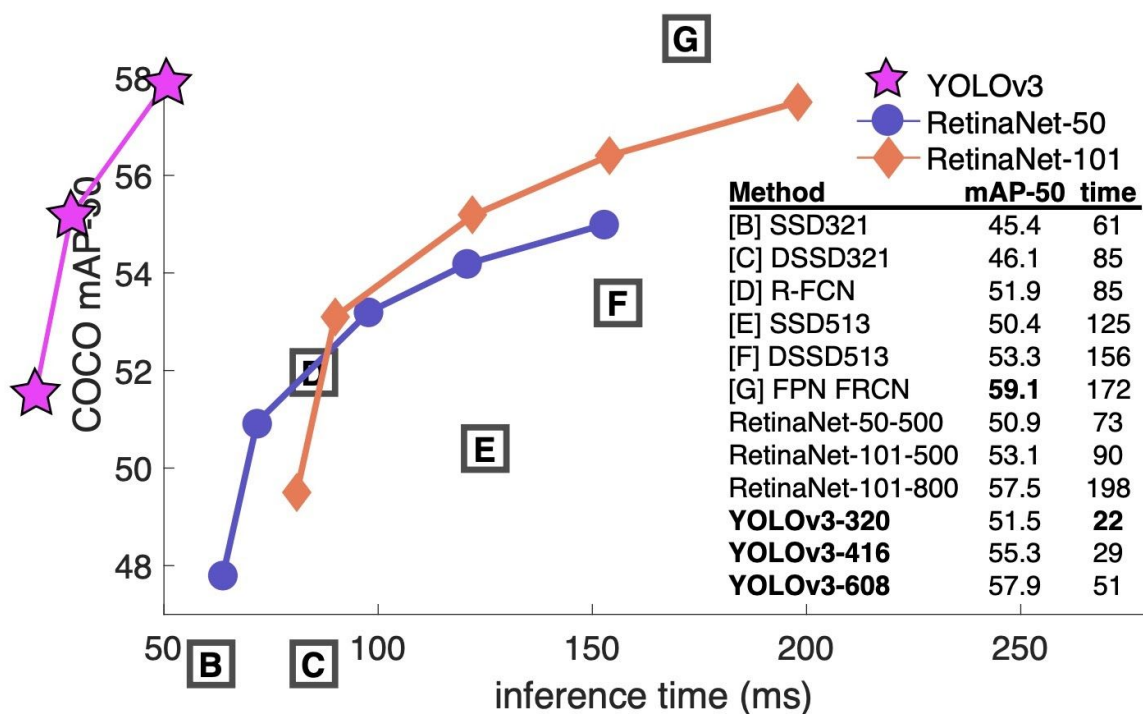


Figure 8. YOLO Performance (YOLOv3 Joseph Redmon)

YOLO is an object detection network which is proven to outperform all the listed similar models. YOLO stands for You Only Look Once and it is widely used by a lot of researchers and developers. YOLO is based on the Darknet framework, a Neural network library written in C and CUDA. YOLO has 3 versions released by its original researchers and authors. Also some other researchers have released YOLO 4 recently. For this research we selected YOLO v3. It is the version 3 release of YOLO model. There are few types of YOLO models like Tiny YOLO but we selected the original YOLO model.

4.2.1.2 Architecture of the Model

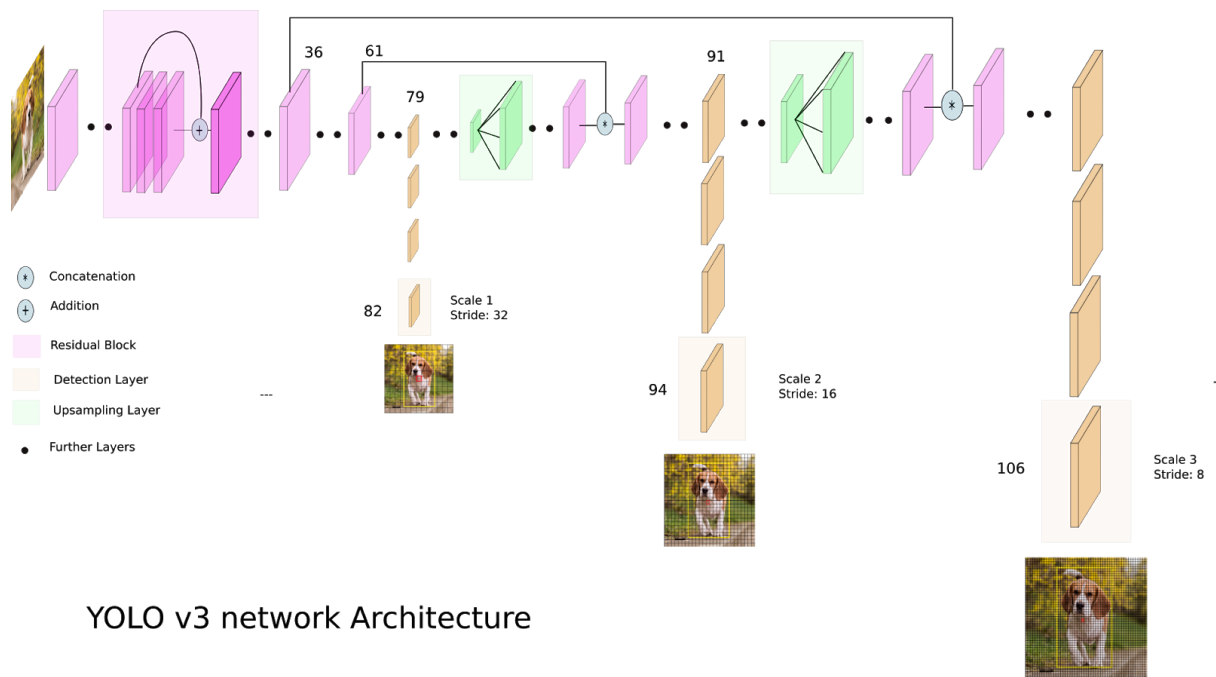


Figure 9. YOLO Architecture (<https://towardsdatascience.com/>)

4.2.1.2.1 Types of Layers

We can identify the following types of layers in YOLO v3 architecture.

1. Residual blocks
2. Detection Layers
3. Upsampling layers

Also there are special layers whereas Addition and Route/Concatenations

4.2.1.2.2 Features of the Model

- Object detections happen in three stages (at different scales as shown in the architecture). Scale size is dependent on the input image size.
- Each grid makes three predictions using three anchor boxes.
- We will get 9 pre defined anchor boxes which are chosen using K mean clustering
- A cell is selected if the center of an object is falls in the receptive fields of the cell

4.2.1.2.3 Output

Equation of output at each cell

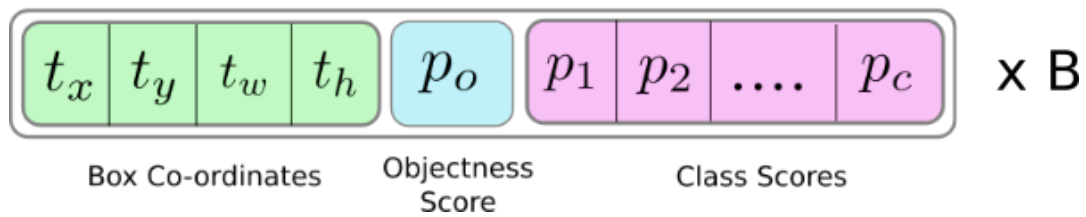


Figure 10. YOLO Output (<https://towardsdatascience.com/>)

$$\# Predictions \text{ per cell} \times (5 + \# \text{ objects to be detected from the data set})$$

B = # Prediction per cell is 3 as defined in the paper

t_x = Center x coordinates of the cell

t_y = Center y coordinates of the cell

t_w = Width of the bounding box

t_h = Height of the bounding box

p_0 = Detection confidence (Objectness Score)

objects to be detected from the data set is the number of objects in the dataset. For example as we use the COCO dataset, it can detect 80 objects. Therefore there are 80 predicted probability values for each and every object that can be detected.

Therefore the output at each scale will be = $3 \times 85 \times \# \text{cells in the scale}$

If three scales are 8×8 , 16×16 and 32×32 (This will depend on the size of the input image)

For 16×16 scale

Output size will be = $3 \times 85 \times 16 \times 16$

And convert this four dimension output to two dimension = 768×85

4.2.1.2.4 Make predictions using outputs

$$b_x = \sigma(t_x) + c_x$$

$$b_y = \sigma(t_y) + c_y$$

$$b_w = p_w e^{t_w}$$

$$b_h = p_h e^{t_h}$$

Above equations have been used to predict the output as mentioned in the YOLO paper. Also to get the output, outputs at various scales have been concatenated together. And use non max suppression to get rid of detections of the same object.

4.2.1.3 Pre Trained YOLO Model

Pre Trained YOLO models comes as two files

1. Model Architecture File (.cfg)
2. Weight file - binary (.weight)

Model architecture file is a human readable file, it contains some metadata related to the model and layers of the network with their attributes.

```
25  [convolutional]
26  batch_normalize=1
27  filters=32
28  size=3
29  stride=1
30  pad=1
31  activation=leaky
32
```

These layers are converted to JSON object using the following algorithm

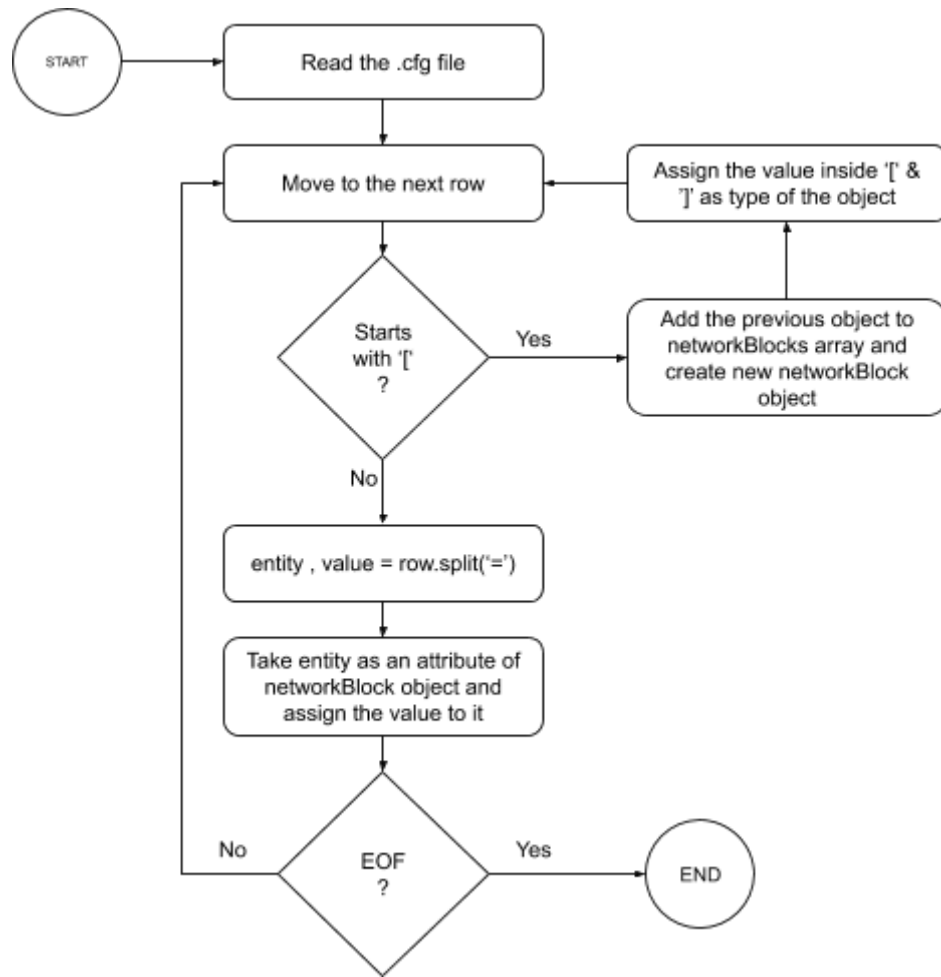


Figure 11. YOLO Model Architecture conversion flowchart

Weight file is a binary file containing an array of Float32 values. The first 4 values are header information, which contains Major version number, Minor Version Number, Subversion number, Images seen. These values are not used for model operations.

Based on the networkBlocks array generated through the above algorithm the weight file is read using the following algorithm.

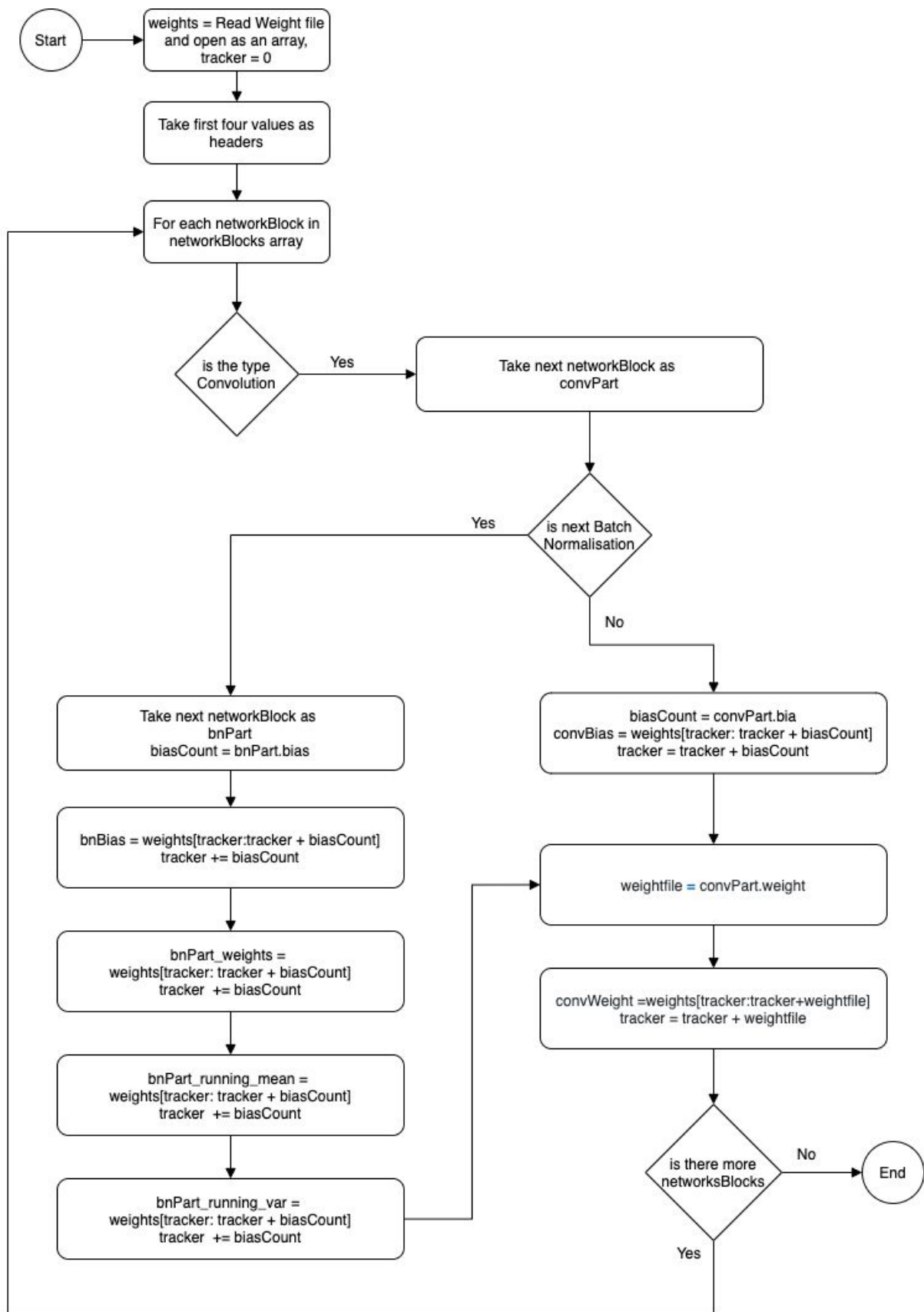


Figure 12. YOLO Model weight reader flowchart

4.2.2 Model Optimizer for ONNX

As introduced before, ONNX (Open Neural Network Exchange) is a way to convert any model type into “onnx” format. As this can be used as an intermediate to convert models, we used an onnx type model as our next model type , Even Though we can convert other types to this format and use it for inference, onnx will become a major dependency of our tool.

We checked whether we can take the weights and architecture details out from an ONNX type model as those data can be used for distribution purposes at inference.

4.2.2.1 Alexnet

For testing purposes we used a pre-trained ONNX model with Alexnet CNN architecture. This neural network has 60 million parameters and 650,000 neurons, consists of five convolutional layers, some of which are followed by max-pooling layers, and three fully-connected layers with a final 1000-way softmax.

4.2.2.2 Alexnet architecture

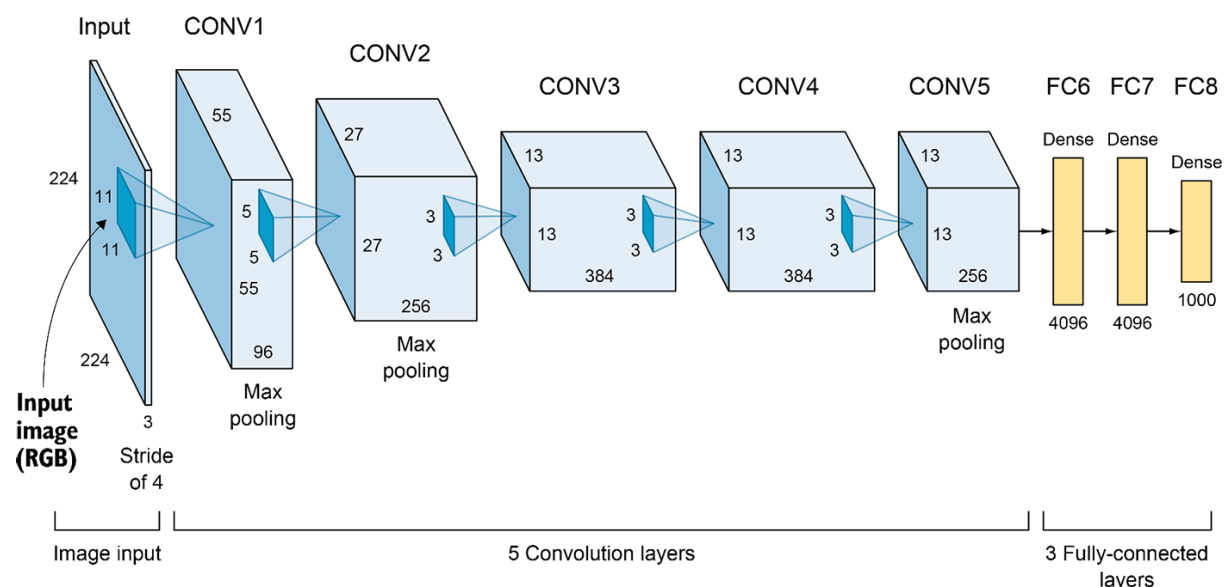


Figure 13. Alexnet Architecture (<https://towardsdatascience.com/>)

4.2.2.3 Steps for reading data inside model

- Model is first loaded using python language.
 - `model = onnx.load("model.onnx")`
- Weight values are read, by iterating through “model.graph.initializer” which consists of tensor proto type objects which includes the data at each node.
- Weight values at each node, which are encoded as “raw_data” should be converted to numeric values with the help of “onnx.numpy_helper”.

4.2.3 Model Optimizer for tensorflow models (.pb file)

A tensorflow model file stores the actual TensorFlow program, or model, and a set of named signatures, each identifying a function that accepts tensor inputs and produces tensor outputs.

Any pre-trained tensorflow models like Inception-V2 model, SDD-MobileNet models have .pb files which include all data about the model. If we make a custom tensorflow model we are able to make a .pb file also. For demonstration purposes we used the MobileNetV1 model here.

In tensorflow there is a tool called “Gfile”, by this platform we are able to directly extract the model data.


```

Name of the node - MobilenetV1/Conv2d_0/weights
Name of the node - MobilenetV1/Conv2d_0/BatchNorm/gamma
Name of the node - MobilenetV1/Conv2d_0/BatchNorm/beta
Name of the node - MobilenetV1/Conv2d_0/BatchNorm/moving_mean
Name of the node - MobilenetV1/Conv2d_0/BatchNorm/moving_variance
Name of the node - MobilenetV1/Conv2d_1_depthwise/depthwise_weights
Name of the node - MobilenetV1/Conv2d_1_depthwise/BatchNorm/gamma
Name of the node - MobilenetV1/Conv2d_1_depthwise/BatchNorm/beta
Name of the node - MobilenetV1/Conv2d_1_depthwise/BatchNorm/moving_mean
Name of the node - MobilenetV1/Conv2d_1_depthwise/BatchNorm/moving_variance
Name of the node - MobilenetV1/Conv2d_1_pointwise/weights
Name of the node - MobilenetV1/Conv2d_1_pointwise/BatchNorm/gamma
Name of the node - MobilenetV1/Conv2d_1_pointwise/BatchNorm/beta
Name of the node - MobilenetV1/Conv2d_1_pointwise/BatchNorm/moving_mean
Name of the node - MobilenetV1/Conv2d_1_pointwise/BatchNorm/moving_variance
Name of the node - MobilenetV1/Conv2d_2_depthwise/depthwise_weights
Name of the node - MobilenetV1/Conv2d_2_depthwise/BatchNorm/gamma
Name of the node - MobilenetV1/Conv2d_2_depthwise/BatchNorm/beta
Name of the node - MobilenetV1/Conv2d_2_depthwise/BatchNorm/moving_mean
Name of the node - MobilenetV1/Conv2d_2_depthwise/BatchNorm/moving_variance
Name of the node - MobilenetV1/Conv2d_2_pointwise/weights
Name of the node - MobilenetV1/Conv2d_2_pointwise/BatchNorm/gamma
Name of the node - MobilenetV1/Conv2d_2_pointwise/BatchNorm/beta
Name of the node - MobilenetV1/Conv2d_2_pointwise/BatchNorm/moving_mean
Name of the node - MobilenetV1/Conv2d_2_pointwise/BatchNorm/moving_variance
Name of the node - MobilenetV1/Conv2d_3_depthwise/depthwise_weights
Name of the node - MobilenetV1/Conv2d_3_depthwise/BatchNorm/gamma
Name of the node - MobilenetV1/Conv2d_3_depthwise/BatchNorm/beta
Name of the node - MobilenetV1/Conv2d_3_depthwise/BatchNorm/moving_mean
Name of the node - MobilenetV1/Conv2d_3_depthwise/BatchNorm/moving_variance
Name of the node - MobilenetV1/Conv2d_3_pointwise/weights
Name of the node - MobilenetV1/Conv2d_3_pointwise/BatchNorm/gamma

```

This is the data about the
convolution layer and FC layer

```

Name of the node - MobilenetV1/Conv2d_13_pointwise/BatchNorm/moving_variance
Value -
[0.53587925 0.5709508 0.50324583 ... 0.53948736 0.5225205 0.5334304 ]
Name of the node - MobilenetV1/Logits/Conv2d_1c_1x1/weights
Value -
[[[-0.00666543 -0.04695416 0.03412621 ... -0.02114701 -0.03981902
0.25771725]
[-0.00442052 0.0182681 -0.05482587 ... -0.04124811 -0.1128666
0.03113037]
[-0.00615936 -0.01368189 0.00865508 ... 0.05865306 -0.08197054
0.08640667]
...
[-0.0047375 0.08308037 0.00153733 ... 0.00461784 0.04064919
-0.00085533]
[-0.0064616 -0.03270431 -0.06865616 ... -0.05149096 0.01180303
0.11459861]
[-0.00453072 0.04251786 -0.06669442 ... 0.00246903 0.05215788
-0.06628785]]]]
Name of the node - MobilenetV1/Logits/Conv2d_1c_1x1/biases
Value -
[-0.678979 -0.11832893 0.07217772 ... -0.43318054 0.07315645
0.3791176 ]
Name of the node - MobilenetV1/Predictions/Reshape/shape
Value -
[ -1 1001]

```

Extract data about the
layer

Figure 14. Console outputs

4.2.3 Building a Raspberry Pi Cluster

To make a raspberry pi cluster, we used two raspberry pis which are the model 3 B + of raspberry pi series. Those raspberry pis have following specifications

- Broadcom BCM2837B0, Cortex-A53 (ARMv8) 64-bit SoC @ 1.4GHz
- 1GB LPDDR2 SDRAM
- 2.4GHz and 5GHz IEEE 802.11.b/g/n/ac wireless LAN, Bluetooth 4.2, BLE

For the data communication switch we used a router which has 4 ethernet ports with WIFI connection to the remote Laptop/PC. Raspberry Pi to Raspberry Pi communication (master to slave/slave to master) happens via a switch. Basic design diagram as follows.

We used ethernet cable communication because wired communication is faster than wireless.

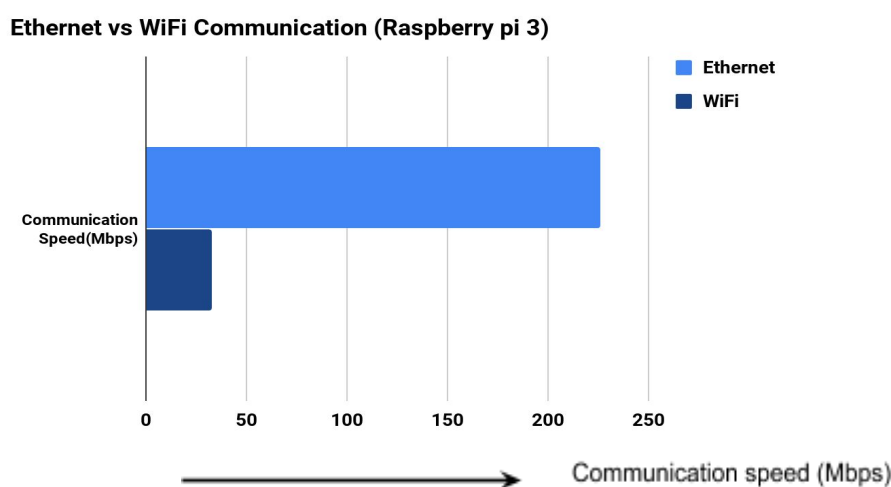


Figure 15. Ethernet vs WIFI speed comparison

As shown in the above figure, Ethernet communication is more efficient than wireless WiFi communication. Therefore we used Ethernet communication for the proposed system.

At the beginning all of the Raspberry Pi nodes are in ideal state. Whenever the input signal is triggered (We used GPIO ports with push buttons to make the input signal) that node becomes the master node. Then the master node starts to send and receive data from the slave nodes.

When we consider the distributed systems, one of the main factors that affects the time taken to complete a task is the communication delay.

Following experiment was done to demonstrate the effect of the communication delay. In this experiment we send image files to the slave nodes for the calculations. Firstly, image data was converted to a string using the base64 encoding. Then the encoded string was sent to the slave device. Below diagram shows the relationship between communication delay vs image size.

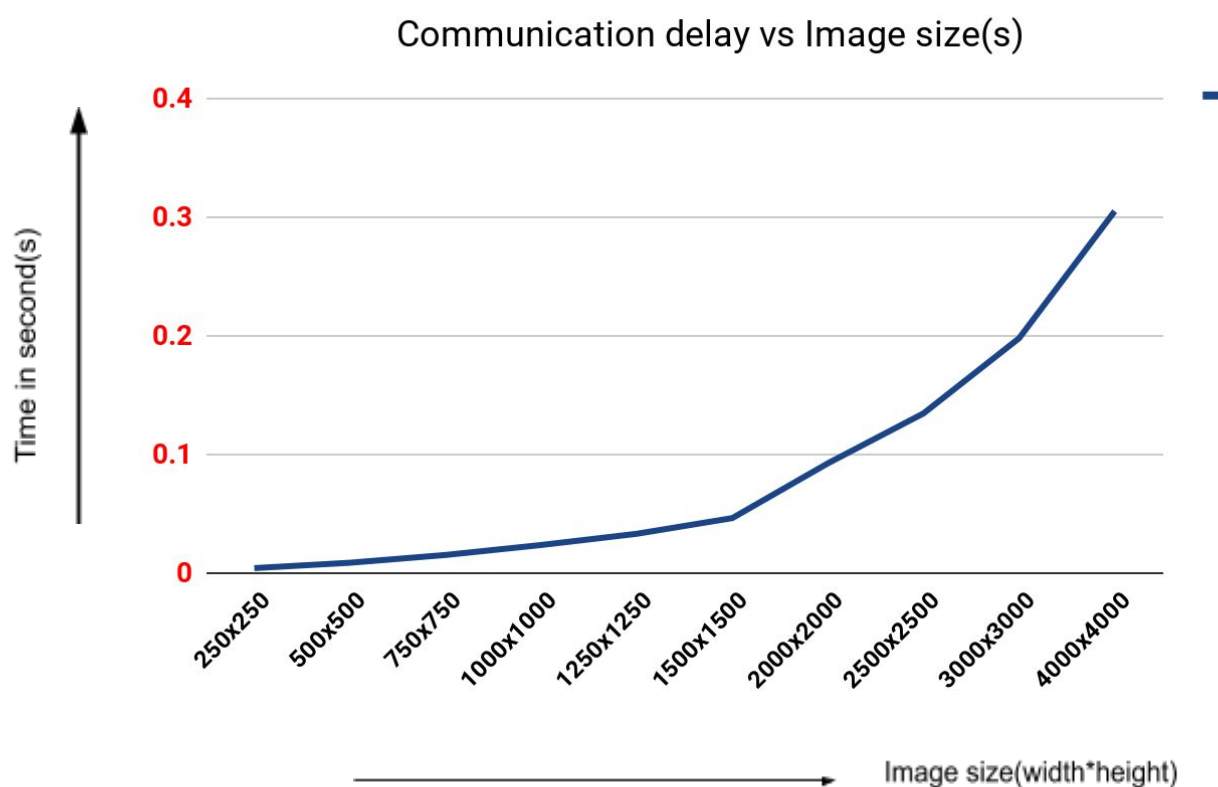


Figure 16. Communication delay vs Image Size

As you see here when the image size increases communication delay also increases.

4.2.4 Distributing a Convolution Layer

Convolution layer is the most time consuming part of the computer vision model. Having several convolution layers may directly affect the inference time. Here we have tried to Distribute the convolution part in several ways. As we aren't able to obtain more than two Raspberry Pis, the experiment was done using only two devices. We can adapt this method to implement the system to work with more than two devices.

4.2.4.1 Method 1

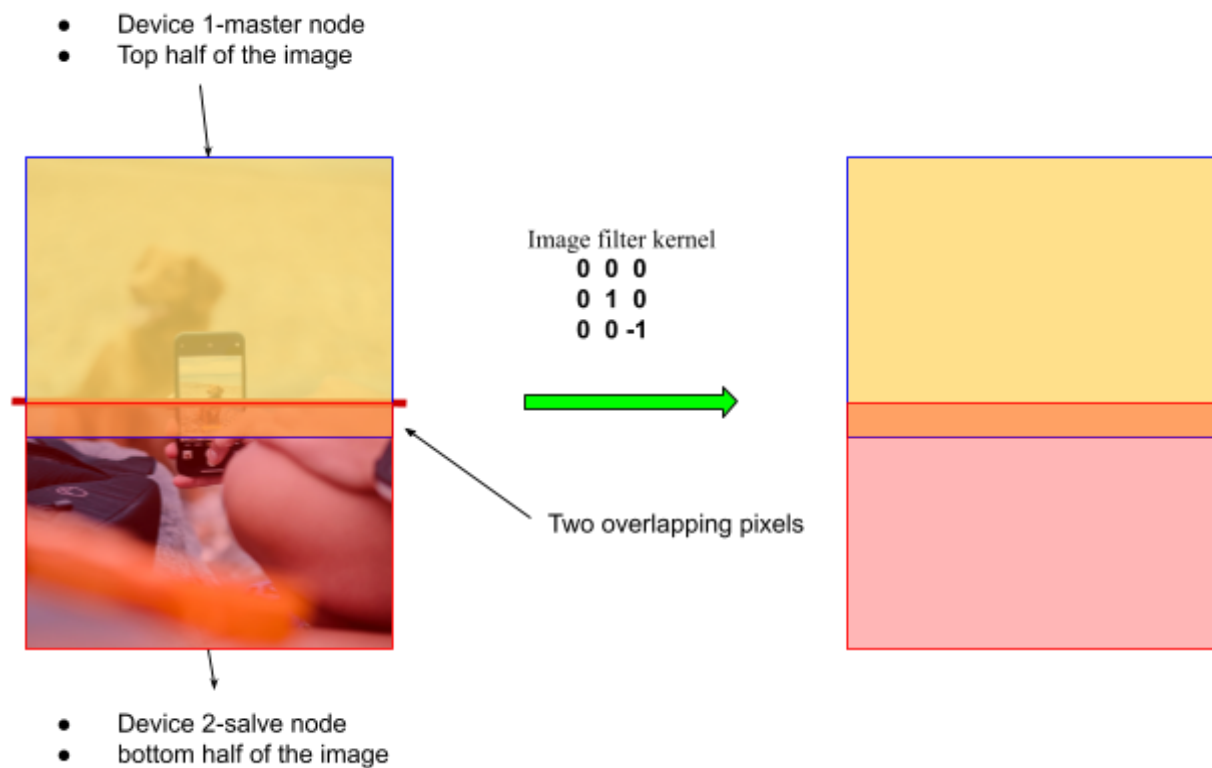


Figure 17. Image partitioning method with overlapping

For the convolution we used following libraries

- Pillow
 - PIL Image
 - Load the image as pillow Image
 - Pillow image crop used to divide the image into two
 - Pillow image paste is used to concat the image
 - PIL ImageFilter
 - For the convolution part
- Base64
 - Convert the image data in to string by b64encode

- Convert the image data string in image by b64decode

Using standard libraries ensures the best performance.

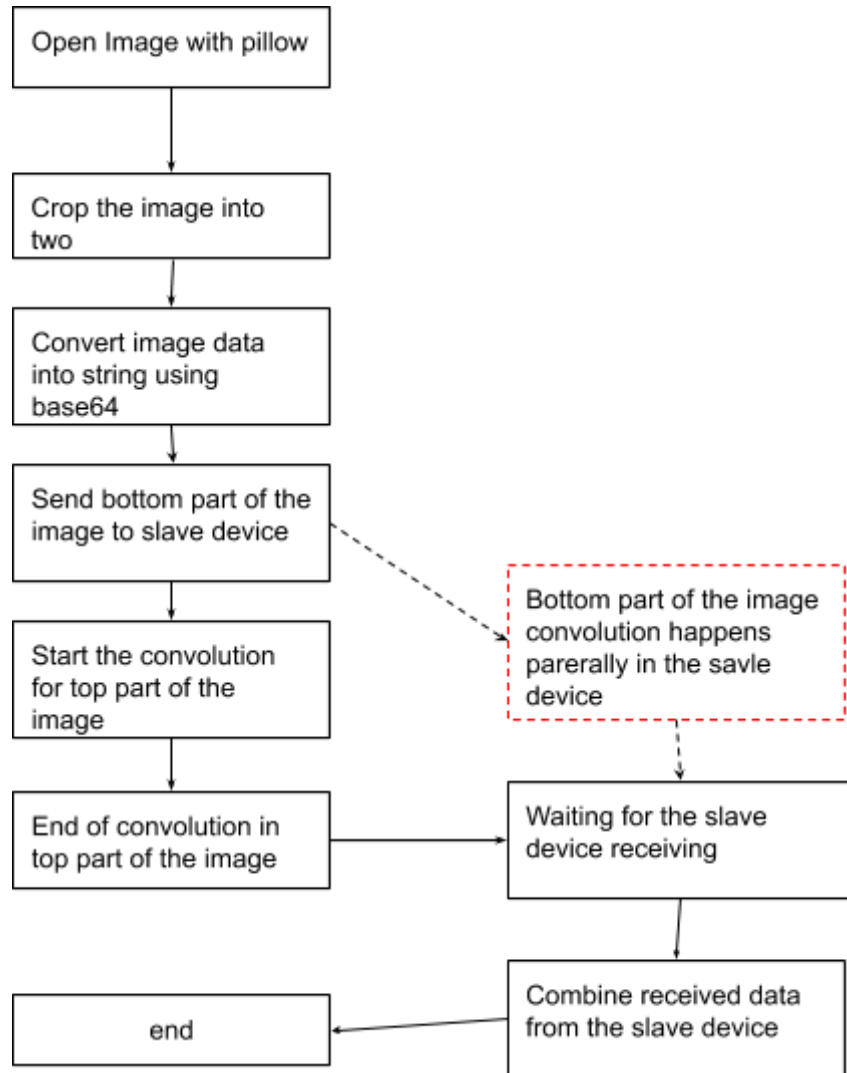


Figure 18. Distributed Single Thread Convolutional operation flowchart

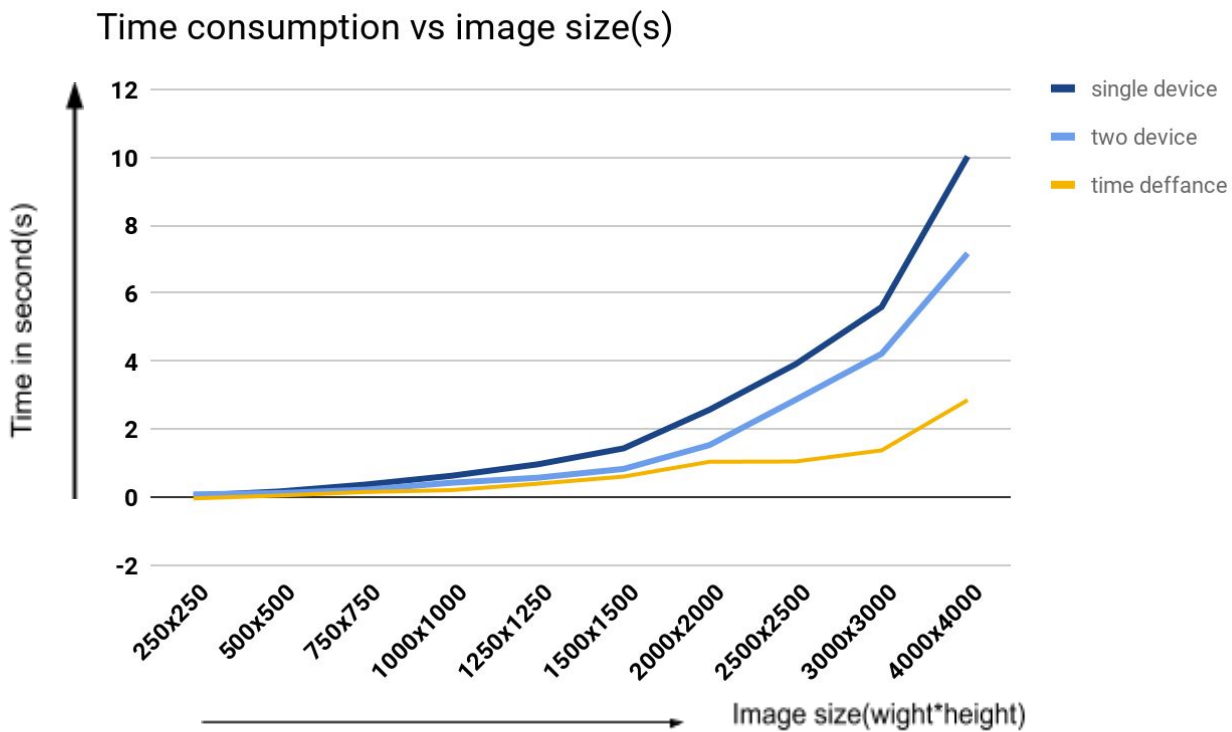


Figure 19. Image resolution vs time consumption

The time consumption for convoluting different sizes of images were calculated and recorded. Test was done three times and the average value was taken.

According to the graph, when the image size is 250x250 or less, a single device performs better than two devices due to communication delays. However when the image resolution gets increased, the distributed system shows more performance than a signal device.

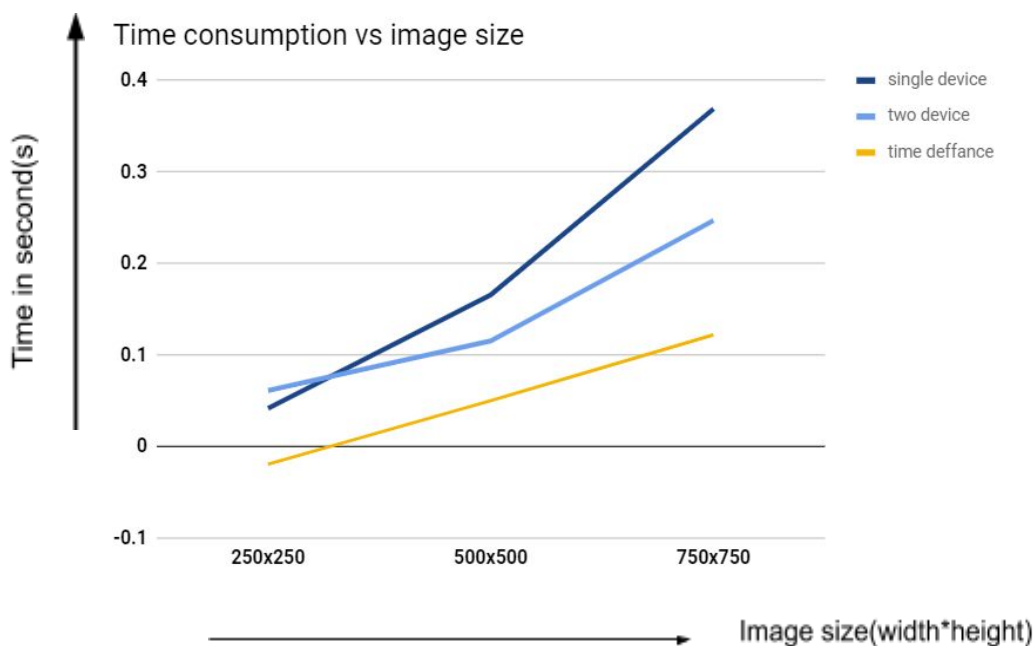


Figure 20. Image resolution vs time consumption zoomed

4.2.4.2 Method 2 : Threaded Distribution

In method 2, we used threads to speed up the process. We identify some operations that can execute parallelly and independently. Here we execute several processes in a secondary thread other than the main thread.

- Converting the bottom part of the image into string using base64
- Send data to the slave devices
- Wait till the slave device sends the convoluted data.

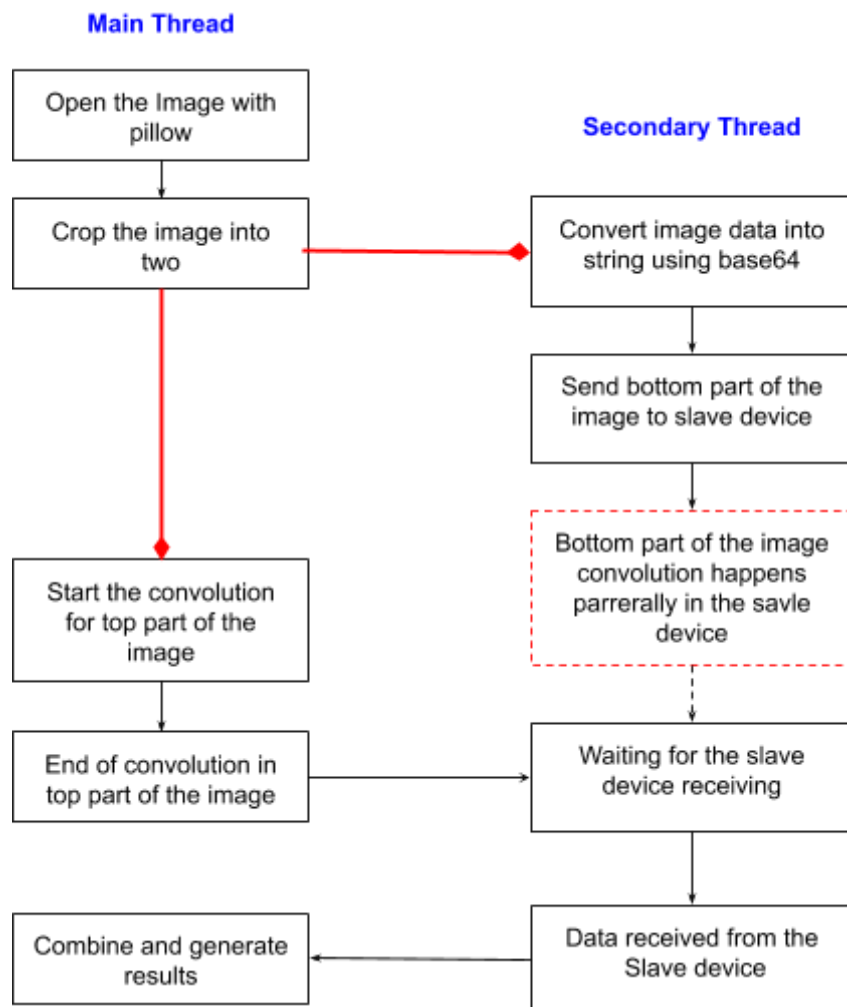


Figure 21. Distributed multi Thread Convolutional operation flowchart

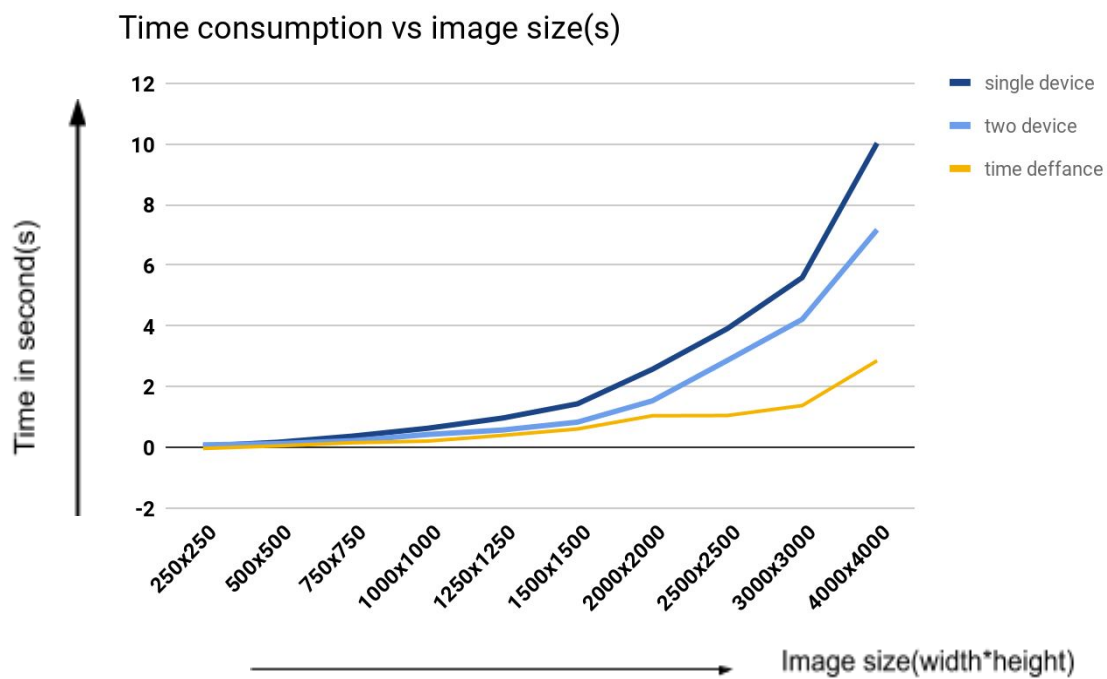


Figure 22. Image resolution vs time consumption

Both method 1 and method 2 do the same work. Only difference is the use of threads in the second method. We can notice slight improvement of process speed in the second method as shown in the below chart.

4.2.4.3 Comparison between method 1 and 2

Let's do the comparison between two methods. If we plot all the diagrams in one chart, (distributed system with and without the threading method) we can not clearly see the difference between those two line (light blue and yellow lines).

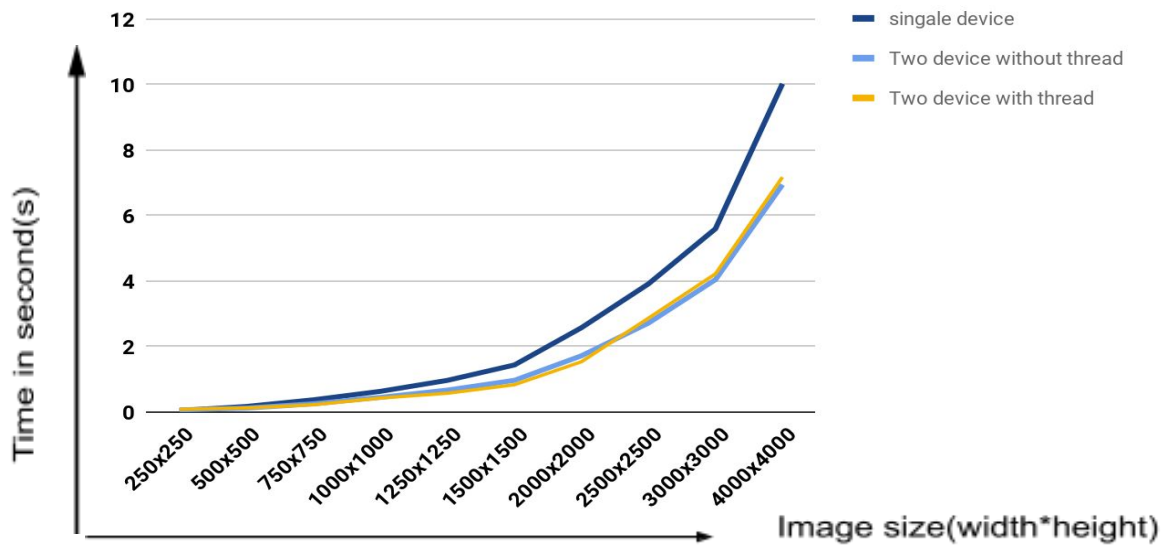


Figure 23. Image resolution vs time consumption single thread and multi thread comparison

However if we zoom into more, at the beginning of 250x250 point, method 2 consumes more time than other methods. However when it comes to the 500x500 point, method 1 and method 2 consume the same time. After passing that point method 2 showing better performance. Therefore In the range of 500x500 - 2500x2500 method 2 shows better performance compared to method 1. According to the experimental results, method 2 shows the best results in higher resolution images.

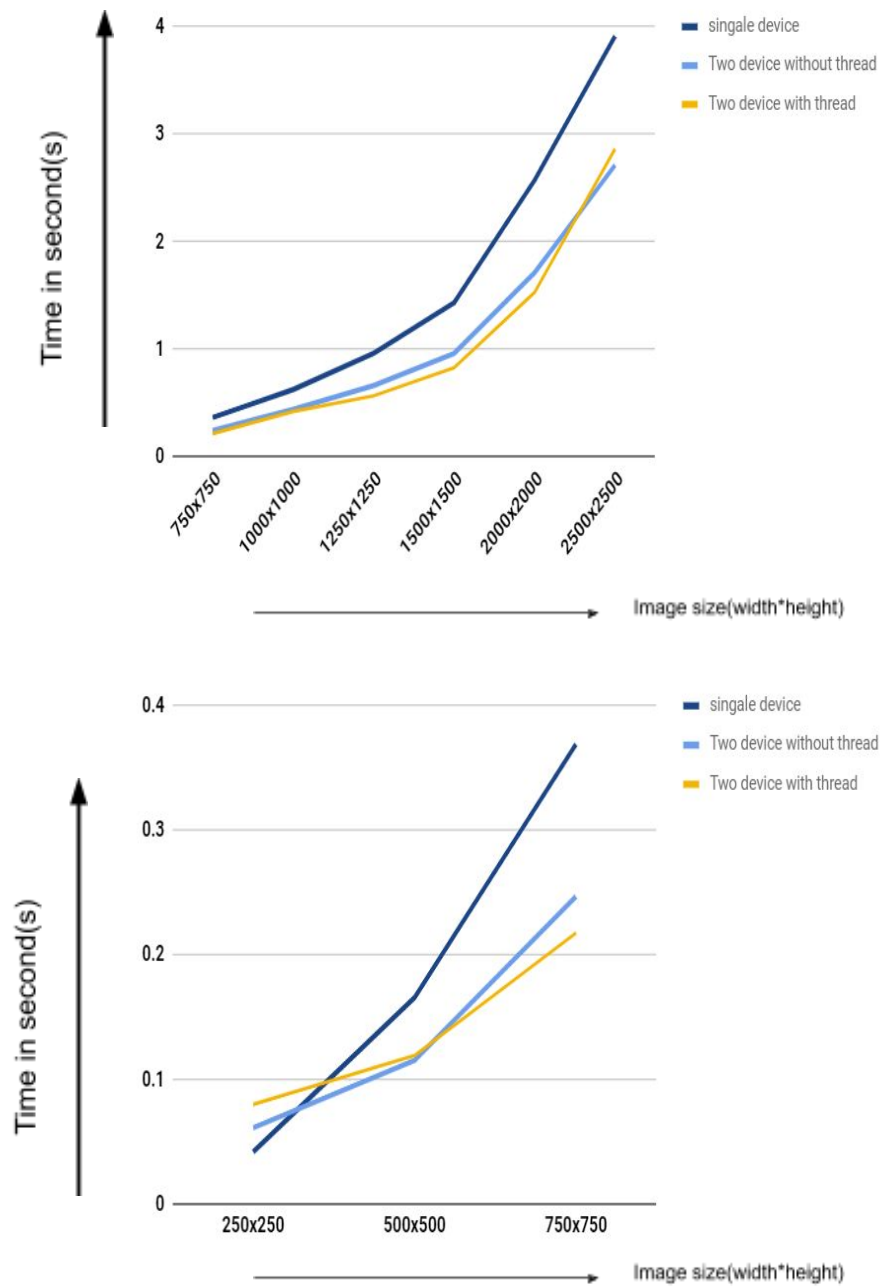


Figure 24. Image resolution vs time consumption conclusion

4. Conclusion/Summary

For now, we have completed upto the following point in the chart below.

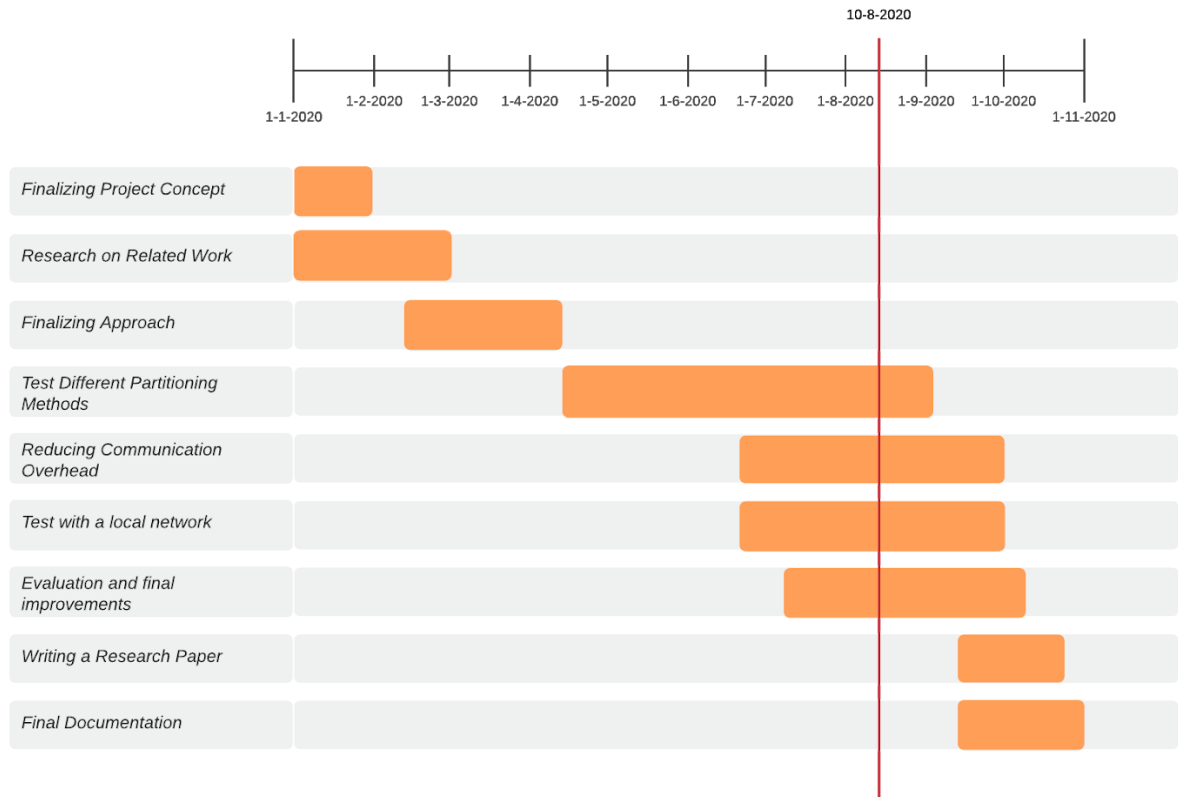


Figure 25. Project Timeline

In conclusion, this project is intended to propose and develop a novel approach that can be used to distributively execute the inference of fully connected neural networks. We hope that this distributed Raspberry Pi based partitioning method will reduce the time taken for inference tasks and make it possible to run large neural networks which usually cannot be deployed to a single device. Furthermore, we plan to investigate the ideal method for partitioning to reduce communication demand among these Raspberry Pis to prevent unwanted communication overheads. We hope this research will help to boost future IoT based and edge computing applications capabilities.

References

- [1] J. Chen and X. Ran, “Deep Learning With Edge Computing: A Review,” *Proc. IEEE*, vol. 107, no. 8, 2019.
- [2] S. Liu, Y. Lin, Z. Zhou, K. Nan, H. Liu, and J. Du, “DeepIoT: Compressing deep neural network structures for sensing systems with a compressor-critic framework,” in *Proc. SenSys*, 2017, pp. 1–4.
- [3] X. Wang, X. Wang, and S. Mao, “RF sensing in the Internet of Things: A general deep learning framework,” *IEEE Commun. Mag.*, vol. 56, no. 9, pp. 62–67, Sep. 2018.
- [4] C. Liu et al., “A new deep learning-based food recognition system for dietary assessment on an edge computing service infrastructure,” *IEEE Trans. Services Comput.*, vol. 11, no. 2, pp. 249–261, Jan. 2018.
- [5] A. H. Jiang et al., “Mainstream: Dynamic stream-sharing for multi-tenant video processing,” in *Proc. USENIX Annu. Tech. Conf. (USENIXATC)*, 2018, pp. 29–42.
- [6] Y. Kang et al., “Neurosurgeon: Collaborative intelligence between the cloud and mobile edge,” *ACM SIGPLAN Notices*, vol. 52, no. 4, pp. 615–629, 2017.
- [7] J. Mao, X. Chen, K. W. Nixon, C. Krieger, and Y. Chen, “MoDNN: Local distributed mobile computing system for deep neural network,” in *Proc. Design, Autom. Test Eur. Conf. Exhibit. (DATE)*, Mar. 2017, pp. 1396–1401.
- [8] Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed adaptive deep learning inference on resource-constrained iot edge clusters,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2348–2359, Nov. 2018.

- [9] S. Teerapittayanon, B. McDanel, and H. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS), Jun. 2017, pp. 328–339.
- [10] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," IEEE Internet of Things Journal (IoT-J), vol. 3, no. 5, pp. 637–646, 2016.
- [11] Leigh Johnson "Portable Computer Vision: TensorFlow 2.0 on a Raspberry Pi" online: <https://towardsdatascience.com/portable-computer-vision-tensorflow-2-0-on-a-raspberry-pi-part-1-of-2-84e318798ce9>.
- [12] R. Stahl, Z. Zhao, D. Mueller-Gritschneider, A. Gerstlauer, and U. Schlichtmann, "Fully distributed deep learning inference on resource-constrained edge devices," in International Conference on Embedded Computer Systems . Springer, 2019, pp. 77–90.
- [13] F. Iandola, "Exploring the Design Space of Deep Convolutional Neural Networks at Large Scale," no. December 2016, 2016.